

Stack Lab (CodeGrade)

 <https://github.com/learn-co-curriculum/python-p3-dsa-stack-lab>  <https://github.com/learn-co-curriculum/python-p3-dsa-stack-lab/issues/new>

Learning Goals

- Implement a **Stack** using a list as the underlying data structure.

Key Vocab

- **Sequence**: a data structure in which data is stored and accessed in a specific order.
- **Stack** is a linear data structure that follows the principle of Last In First Out (LIFO)
- **Index**: the location, represented by an integer, of an element in a sequence.
- **Iterable**: able to be broken down into smaller parts of equal size that can be processed in turn. You can loop through any iterable object.
- **Slice**: a group of neighboring elements in a sequence.
- **List**: a mutable data type in Python that can store many types of data. The most common data structure in Python.
- **Tuple**: an immutable data type in Python that can store many types of data.
- **Range**: a data type in Python that stores integers in a fixed pattern.
- **String**: an immutable data type in Python that stores unicode characters in a fixed pattern. Iterable and indexed, just like other sequences.

Instructions

In the previous lesson, you learned what a **Stack** is and what methods they commonly include. In this lab, you will be building out an implementation of a **Stack**. You will be using a list as the underlying data structure, and calling on some built-in Python list methods to build

your `Stack` class's functionality.

Start by forking and cloning this lab. You'll be writing your code in the `lib/stack.py` file. You can run the tests at any point using `pytest -x` to check your work.

Bonus

If you'd like an extra challenge, try implementing the additional functionality below. There are tests for these in the `testing/stack_test.py` file; uncomment the **bonus methods** section in the test file to try these out.

1. Modify your `Stack __init__()` method to take an optional `limit` value and set that as an attribute.
2. Update the `Stack push()` value to only push the passed-in value if there's still room in the `Stack`. If the `Stack` is full, the method should throw an error.
3. Implement the following additional methods:
 - `Stack size()` : returns the number of elements contained in the `Stack`
 - `Stack empty()` : returns true if the `Stack` is empty; false otherwise
 - `Stack full()` : returns true if the `Stack` is full; false otherwise
 - `Stack search(value)` : returns the distance between the top of the stack and the target element if it's present; -1 otherwise. If the target element is at the top of the stack your code should return 0.

After you've made these changes, you might want to take another look through your code and see if there's any refactoring you can do.

Conclusion

In this lesson, we got some practice building a data structure from scratch by implementing a `Stack` class. Recall that the runtime of our data structure will depend on what data structure it uses under the hood. For this lab, we used a list as the underlying data structure, which means the runtime for the `search()` method is $O(n)$, and the runtime for all of the other methods is $O(1)$.

Resources

- [Wikipedia: Stack \(abstract data type\)](https://en.wikipedia.org/wiki/Stack_(abstract_data_type))  [_\(https://en.wikipedia.org/wiki/Stack_\(abstract_data_type\)\)](https://en.wikipedia.org/wiki/Stack_(abstract_data_type)).

This tool needs to be loaded in a new browser window

Load Stack Lab (CodeGrade) in a new window